

使用預先訓練的 TensorFlow.js 機器學習模型，以 JavaScript 建立智慧型網路攝影機

來源：<https://codelabs.developers.google.com/codelabs/tensorflowjs-object-detection?hl=zh-tw>

步驟數：9

在本程式碼研究室中，您將瞭解如何載入及使用其中一種 TensorFlow.js 預先訓練模型 (COCO-SSD)，並使用該模型來辨識訓練過的常見物件。

目錄

1. [事前準備](#)
2. [什麼是 TensorFlow.js ?](#)
3. [預先訓練模型](#)
4. [做好準備](#)
5. [填入 HTML 架構](#)
6. [新增樣式](#)
7. [建立 JavaScript 骨架](#)
8. [機器學習模型使用情況](#)
9. [恭喜](#)

步驟 1 · 約 2 分鐘

1. 事前準備

機器學習是近來相當熱門的詞彙。這項技術的應用範圍似乎沒有限制，而且近期內可望觸及幾乎所有產業。如果您是工程師或設計人員，無論是前端或後端，且熟悉 JavaScript，這個程式碼研究室可協助您開始在技能組合中加入機器學習。

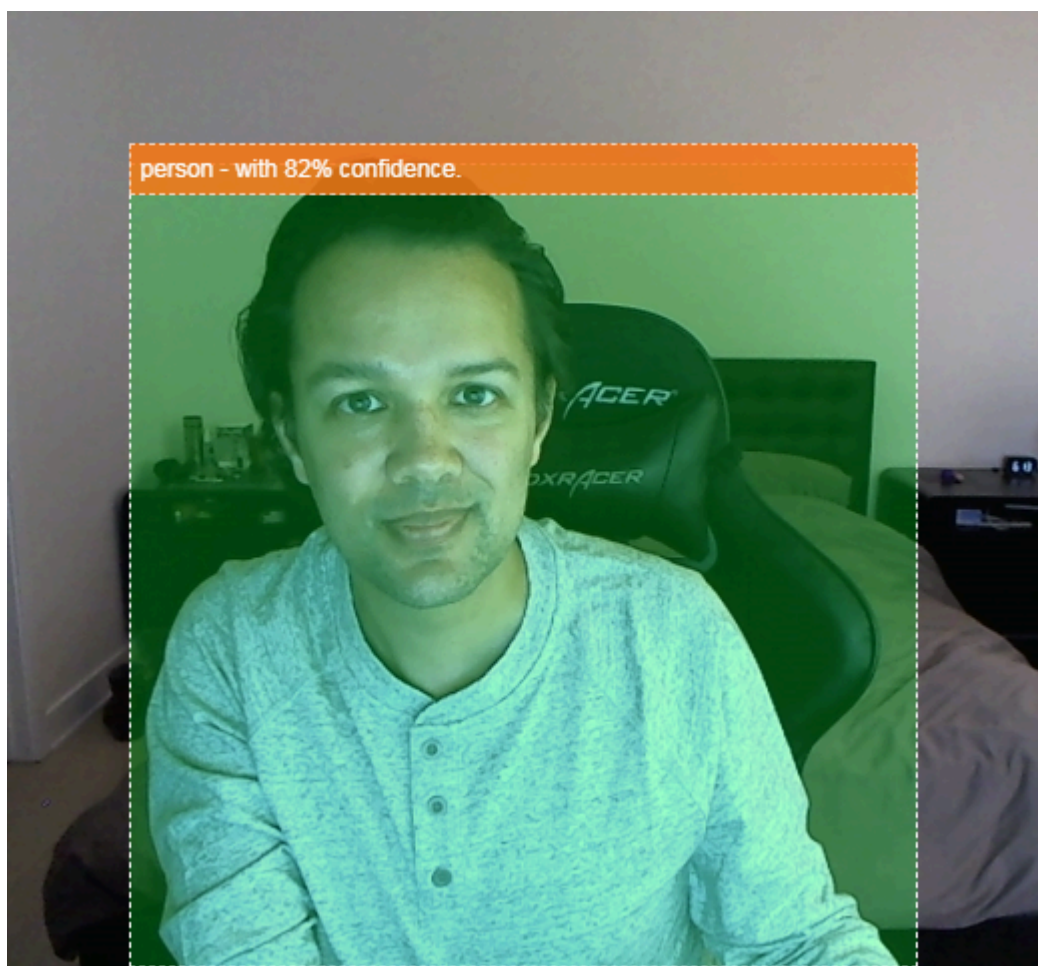
必要條件

本程式碼研究室適合熟悉 JavaScript 的資深工程師。

建構項目

在本程式碼研究室中，您將

- 建立網頁，透過 TensorFlow.js 直接在網路瀏覽器中使用機器學習技術，從網路攝影機串流分類及偵測常見物件 (沒錯，包括一次偵測多個物件)。
- 運用一般網路攝影機辨識物體，並取得每個物體的定界框座標
- 在影片串流中醒目顯示找到的物件，如下所示：



想像一下，如果能夠偵測影片中是否有人，就能計算特定時間的人數，估算特定區域一整天的忙碌程度，或是在偵測到狗狗出現在家中某個不該出現的房間時，傳送快訊給你。如果可以做到這點，您就能自行製作 [Google Nest Cam](#)，並在偵測到任何類型的入侵者時收到警報！很方便吧？會不會很麻煩？不對，開始駭客任務吧！

課程內容

- 如何載入預先訓練的 TensorFlow.js 模型。
- 如何從即時網路攝影機串流擷取資料，並繪製到畫布上。

- 如何分類圖片影格，找出模型已訓練辨識的任何物件定界框。
- 如何使用從模型傳回的資料，醒目顯示找到的物件。

本程式碼實驗室著重於說明如何開始使用 TensorFlow.js 預先訓練模型。不會說明與 TensorFlow.js 和機器學習無關的概念和程式碼區塊，但會事先準備好這些程式碼區塊，屆時您只要複製及貼上即可。

2. 什麼是 TensorFlow.js ?



TensorFlow.js 是開放原始碼機器學習程式庫，可在 JavaScript 執行的任何位置執行。這個程式庫是以原始的 **Python 版 TensorFlow** 程式庫為基礎，目標是為 JavaScript 生態系統重新建立這種開發人員體驗和 API 集。

哪些類別可以套用顯示設定？

由於 JavaScript 具有可攜性，您現在只要使用 1 種語言，就能輕鬆在下列所有平台執行機器學習：

- 網頁瀏覽器中的用戶端，使用原生 JavaScript
- 伺服器端，甚至是 **Raspberry Pi** 等物聯網裝置 (使用 Node.js)
- 使用 Electron 的電腦應用程式
- 使用 React Native 的原生行動應用程式

TensorFlow.js 也支援這些環境中的多個後端 (例如 CPU 或 WebGL 等實際硬體環境)。在此情況下，「後端」並非指伺服器端環境，而是指執行後端 (例如 WebGL 中的用戶端)，確保相容性並維持快速運作。TensorFlow.js 目前支援：

- **在裝置的顯示卡 (GPU) 上執行 WebGL**：這是執行較大型模型 (超過 3 MB) 的最快方式，可透過 GPU 加速。
- **在 CPU 上執行 Web Assembly (WASM)** - 提升各種裝置的 CPU 效能，包括舊款手機。這類模型較小 (小於 3 MB)，由於將內容上傳至圖形處理器的負擔，這類模型在 CPU 上使用 WASM 執行的速度，實際上會比使用 WebGL 更快。
- **CPU 執行** - 如果沒有其他環境可用，則應採用這個備援方案。這是三者中最慢的，但隨時可供使用。

注意：如果您知道要執行的裝置，可以選擇**強制使用其中一個後端**，也可以不指定，讓 TensorFlow.js 為您決定。

用戶端超能力

在用戶端電腦的網路瀏覽器中執行 TensorFlow.js，可帶來多項值得考慮的優點。

隱私權

您可以在用戶端電腦上訓練及分類資料，完全不必將資料傳送至第三方網路伺服器。有時，這可能是為了遵守當地法律 (例如 GDPR) 的規定，或是處理使用者想保留在電腦上，不想傳送給第三方的資料。

速度

由於不必將資料傳送至遠端伺服器，推論 (分類資料的行為) 速度會更快。更棒的是，如果使用者授予存取權，您就能直接存取裝置的感應器，例如相機、麥克風、GPS、加速度計等。

觸及率和規模

只要按一下連結，世界各地的使用者就能在瀏覽器中開啟網頁，並使用您製作的內容。您不必為了使用機器學習系統，在伺服器端進行複雜的 Linux 設定，包括 CUDA 驅動程式等。

費用

由於沒有伺服器，您只需支付 CDN 費用，即可代管 HTML、CSS、JS 和模型檔案。與讓伺服器 (可能附有顯示卡) 全天候運作相比，CDN 的成本便宜許多。

伺服器端功能

運用 Node.js 實作的 TensorFlow.js 可啟用下列功能。

完整支援 CUDA

在伺服器端，如要使用顯示卡加速，必須安裝 [NVIDIA CUDA 驅動程式](#)，才能讓 TensorFlow 與顯示卡搭配運作 (與使用 WebGL 的瀏覽器不同，不需要安裝)。不過，如果完全支援 CUDA，就能充分運用顯示卡的低階功能，進而縮短訓練和推論時間。由於兩者共用相同的 C++ 後端，因此效能與 Python TensorFlow 實作項目相同。

模型大小

如果是研究領域的尖端模型，您可能會使用非常大的模型，大小可能達到 GB 級。由於每個瀏覽器分頁的記憶體用量有限制，目前無法在網路瀏覽器中執行這些模型。如要執行這些較大的模型，您可以在自己的伺服器上使用 Node.js，並具備有效執行這類模型所需的硬體規格。

IOT

Node.js 支援 Raspberry Pi 等熱門單板電腦，因此您也可以在这些裝置上執行 TensorFlow.js 模型。

速度

Node.js 是以 JavaScript 編寫而成，因此可從即時編譯中獲益。這表示使用 Node.js 時，您通常會發現效能有所提升，因為系統會在執行階段進行最佳化，特別是針對您可能執行的任何前置處理。如需這方面的絕佳範例，請參閱[這份個案研究](#)，瞭解 Hugging Face 如何使用 Node.js，將自然語言處理模型的效能提升一倍。

現在您已瞭解 TensorFlow.js 的基本概念、執行位置和一些優點，接下來就開始使用這項技術完成實用工作吧！

3. 預先訓練模型

TensorFlow.js 提供各種預先訓練的機器學習 (ML) 模型。這些模型是由 TensorFlow.js 團隊訓練，並封裝在易於使用的類別中，是開始使用機器學習的絕佳方式。不必建立及訓練模型來解決問題，您可以匯入預先訓練的模型做為起點。

您可以在 TensorFlow.js 的「[Models for JavaScript](#)」頁面中，找到越來越多易於使用的預先訓練模型。您也可以從其他地方取得可在 TensorFlow.js 中運作的轉換 TensorFlow 模型，包括 [TensorFlow Hub](#)。

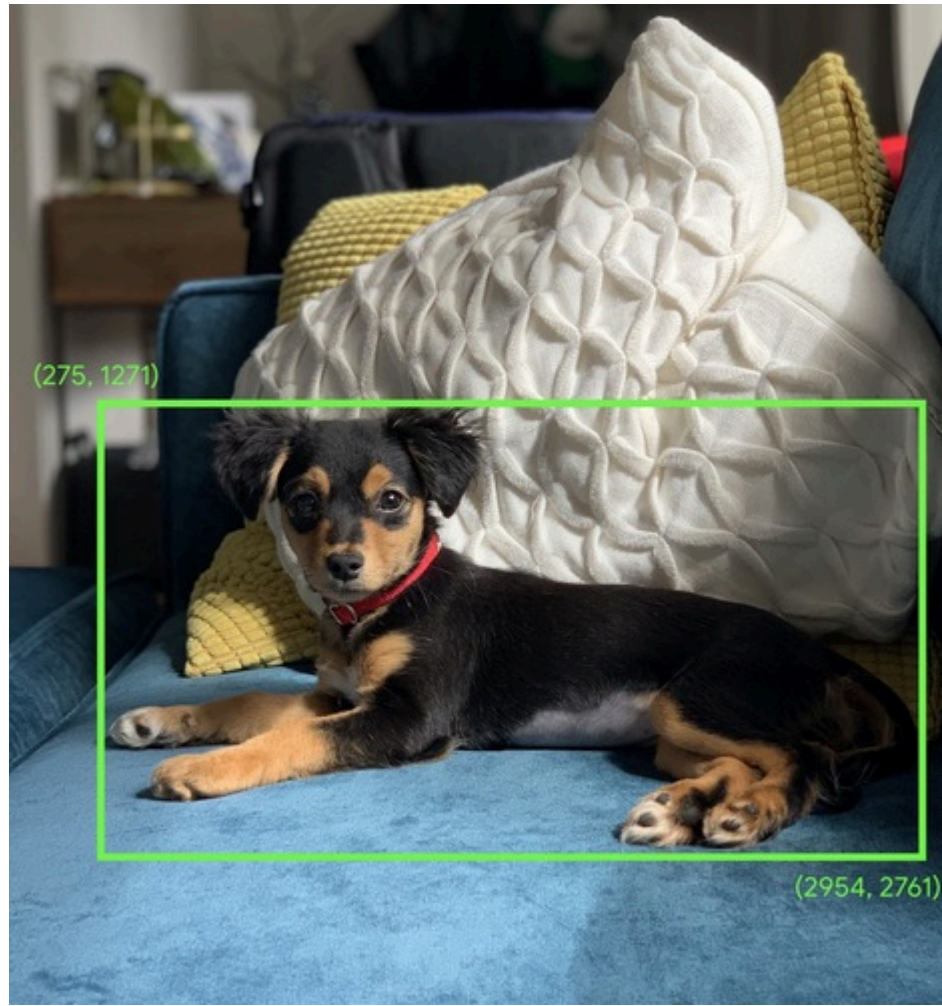
為什麼要使用預先訓練模型？

如果熱門的預先訓練模型符合您的用途，從這類模型著手有許多好處，例如：

1. **不必自行收集訓練資料。**準備格式正確的資料並加上標籤，讓機器學習系統從中學習，可能非常耗時且所費不貲。
2. **以更少的成本和時間，快速設計構想原型。**
如果預先訓練模型足以滿足需求，就沒有必要「重新發明輪子」，您可以專注於運用模型提供的知識，實現創意構想。
3. **運用最先進的研究成果。**預先訓練模型通常以熱門研究為基礎，因此您能接觸這類模型，同時瞭解模型在現實世界中的效能。
4. **簡單易用，且提供豐富的文件。**這類模型非常熱門。
5. **遷移學習 功能。**部分預先訓練模型提供遷移學習功能，也就是將從一項機器學習工作學到的資訊，轉移到另一個類似範例的實務做法。舉例來說，如果提供新的訓練資料，原本訓練辨識貓的模型就能重新訓練，用來辨識狗。由於不必從空白畫布開始，因此速度會更快。模型可以運用已學會的貓辨識能力，辨識新事物。畢竟狗也有眼睛和耳朵，因此如果模型已知道如何找出這些特徵，我們就完成一半了。以更快的速度，使用自己的資料重新訓練模型。

什麼是 COCO-SSD ？

COCO-SSD 是預先訓練的物件偵測機器學習模型名稱，您將在本程式碼研究室中使用這個模型，在單一圖片中定位及辨識多個物件。換句話說，模型會找出經過訓練可辨識的物件，並提供[定界框](#)，指出物件在您提供的圖片中的位置。如下圖所示：



如果上圖中不只一隻狗，系統會提供 2 個定界框的座標，說明每隻狗的位置。COCO-SSD 經過預先訓練，可辨識 90 種常見日常物件，例如人、車、貓等。

這個名稱的由來為何？

這個名稱聽起來可能很奇怪，但其實是由 2 個縮寫字組成：

- COCO：是指模型是使用 [COCO \(Common Objects in Context\) 資料集](#) 訓練而成，任何人都可以免費下載及使用這個資料集來訓練自己的模型。這個資料集包含超過 20 萬張已加上標籤的圖片，可用於學習。
- SSD (Single Shot MultiBox Detection)：指模型實作中使用的模型架構部分。您不需要瞭解這項資訊才能完成程式碼實驗室，但如果您有興趣，可以按[這裡](#)進一步瞭解 SSD：固態硬碟。

步驟 4 · 約 5 分鐘

4. 做好準備

軟硬體需求

- 新式網路瀏覽器。
- 具備 HTML、CSS、JavaScript 和 [Chrome 開發人員工具](#) (查看控制台輸出內容) 的基本知識。

開始編寫程式碼

我們已為 [Glitch.com](#) 或 [Codepen.io](#) 建立可做為起點的樣板範本。您只需按一下滑鼠，即可複製任一範本，做為本程式碼研究室的基礎狀態。

在 Glitch 上，按一下「remix this」按鈕即可分支，並建立可編輯的新檔案集。

或者，在 Codepen 上，按一下畫面右下角的「fork」。

這個非常簡單的架構會提供下列檔案：

- HTML 網頁 (index.html)
- 樣式表 (style.css)
- 用於編寫 JavaScript 程式碼的檔案 (script.js)

為方便起見，HTML 檔案中已新增 TensorFlow.js 程式庫的匯入項目。這是訂閱按鈕的圖示：

index.html

```
<!-- Import TensorFlow.js library -->
<script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs/dist/tf.min.js"
type="text/javascript"></script>
```

替代做法：使用偏好的網頁編輯器或在本機工作

如要下載程式碼並在本機或使用其他線上編輯器作業，只要在相同目錄中建立上述 3 個檔案，然後將 Glitch 樣板中的程式碼複製並貼到每個檔案即可。

注意：如果您是在本機執行，而非 Codepen.io 或類似網站，請務必透過網路伺服器提供檔案，而非只是按兩下 index.html，透過 file:// 通訊協定開啟檔案。

5. 填入 HTML 架構

所有原型都需要一些基本的 HTML 支架。稍後您會使用這項功能，算繪機器學習模型的輸出內容。現在就來設定：

- 網頁標題
- 一些說明文字
- 啟用網路攝影機的按鈕
- 用於算繪網路攝影機串流的影片標記

如要設定這些功能，請開啟 index.html，並將現有程式碼貼到下列程式碼上：

index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Multiple object detection using pre trained model in TensorFlow.js</title>
    <meta charset="utf-8">
    <!-- Import the webpage's stylesheet -->
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <h1>Multiple object detection using pre trained model in TensorFlow.js</h1>

    <p>Wait for the model to load before clicking the button to enable the webcam - at which
    point it will become visible to use.</p>

    <section id="demos" class="invisible">

      <p>Hold some objects up close to your webcam to get a real-time classification! When
      ready click "enable webcam" below and accept access to the webcam when the browser asks
      (check the top left of your window)</p>

      <div id="liveView" class="camView">
        <button id="webcamButton">Enable Webcam</button>
        <video id="webcam" autoplay muted width="640" height="480"></video>
      </div>
    </section>

    <!-- Import TensorFlow.js library -->
    <script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs/dist/tf.min.js"
    type="text/javascript"></script>
    <!-- Load the coco-ssd model to use to recognize things in images -->
    <script src="https://cdn.jsdelivr.net/npm/@tensorflow-models/coco-ssd"></script>

    <!-- Import the page's JavaScript to do some stuff -->
    <script src="script.js" defer></script>
  </body>
</html>
```

瞭解程式碼

請注意您新增的幾項重要內容：

- 您加入了 `<h1>` 標記和一些標頭的 `<p>` 標記，以及網頁使用方式的相關資訊。這裡沒有什麼特別之處。

您也新增了代表展示空間的區段標記：

index.html

```
<section id="demos" class="invisible">

  <p>Hold some objects up close to your webcam to get a real-time classification! When
  ready click "enable webcam" below and accept access to the webcam when the browser asks
  (check the top left of your window)</p>

  <div id="liveView" class="webcam">
    <button id="webcamButton">Enable Webcam</button>
    <video id="webcam" autoplay width="640" height="480"></video>
  </div>
</section>
```

- 一開始，您會為這個 `section` 提供「invisible」類別。這樣一來，您就能向使用者清楚說明模型何時準備就緒，以及何時可安全點選「啟用網路攝影機」按鈕。
- 您已新增「啟用網路攝影機」按鈕，您將在 CSS 中設定樣式。
- 您也新增了影片標記，要將網路攝影機輸入內容串流至該標記。您很快就會在 JavaScript 程式碼中設定這項功能。

如果現在預覽輸出內容，畫面應如下所示：

Multiple object detection using pre trained model in TensorFlow.js

Wait for the model to load before clicking the button to enable the webcam - at which point it will become visible to use.

Hold some objects up close to your webcam to get a real-time classification! When ready click "enable webcam" below and accept access to the webcam when the browser asks (check the top left of your window)

Enable Webcam

提示：如要在 Glitch.com 上即時預覽作品，請按一下畫面左上方的「show」下拉式選單，然後選取程式碼旁的。

步驟 6 · 約 5 分鐘

6. 新增樣式

元素預設值

首先，請為剛才新增的 HTML 元素新增樣式，確保這些元素能正確算繪：

style.css

```
body {
  font-family: helvetica, arial, sans-serif;
  margin: 2em;
  color: #3D3D3D;
}

h1 {
  font-style: italic;
  color: #FF6F00;
}

video {
  display: block;
}

section {
  opacity: 1;
  transition: opacity 500ms ease-in-out;
}
```

接著，新增一些實用的 CSS 類別，協助處理使用者介面的各種不同狀態，例如要隱藏按鈕，或在模型尚未準備就緒時，讓示範區域顯示為無法使用。

style.css

```

.removed {
  display: none;
}

.invisible {
  opacity: 0.2;
}

.camView {
  position: relative;
  float: left;
  width: calc(100% - 20px);
  margin: 10px;
  cursor: pointer;
}

.camView p {
  position: absolute;
  padding: 5px;
  background-color: rgba(255, 111, 0, 0.85);
  color: #FFF;
  border: 1px dashed rgba(255, 255, 255, 0.7);
  z-index: 2;
  font-size: 12px;
}

.highlighter {
  background: rgba(0, 255, 0, 0.25);
  border: 1px dashed #fff;
  z-index: 1;
  position: absolute;
}

```

太好了！就是這麼簡單。如果您已使用上述 2 段程式碼成功覆寫樣式，即時預覽畫面現在應如下所示：

Multiple object detection using pre trained model in TensorFlow.js

Wait for the model to load before clicking the button to enable the webcam - at which point it will become visible to use.

Hold some objects up close to your webcam to get a real-time classification!
When ready click "enable webcam" below and accept access to the webcam when the browser asks (check the top left of your window)

Enable Webcam

請注意，由於 HTML 預設套用「invisible」類別，因此示範區域文字和按鈕無法使用。模型準備就緒後，您將使用 JavaScript 移除這個類別。

7. 建立 JavaScript 骨架

提示：下方程式碼依序顯示，但為了深入說明，我們將其分成較小的部分。如果您每次都將每個區隔複製並貼到 JavaScript 檔案結尾，或是按照建議取代上一個步驟中定義的空白函式，一切應該都能正常運作 :-)

參照重要 DOM 元素

首先，請確認您可以存取網頁的重要部分，稍後您將在程式碼中操控或存取這些部分：

script.js

```
const video = document.getElementById('webcam');
const liveView = document.getElementById('liveView');
const demosSection = document.getElementById('demos');
const enableWebcamButton = document.getElementById('webcamButton');
```

確認是否支援網路攝影機

您現在可以新增一些輔助函式，檢查目前使用的瀏覽器是否支援透過 `getUserMedia` 存取網路攝影機串流：

script.js

```
// Check if webcam access is supported.
function getUserMediaSupported() {
  return !(navigator.mediaDevices &&
    navigator.mediaDevices.getUserMedia);
}

// If webcam supported, add event listener to button for when user
// wants to activate it to call enableCam function which we will
// define in the next step.
if (getUserMediaSupported()) {
  enableWebcamButton.addEventListener('click', enableCam);
} else {
  console.warn('getUserMedia() is not supported by your browser');
}

// Placeholder function for next step. Paste over this in the next step.
function enableCam(event) {
}
```

擷取網路攝影機串流

接著，複製並貼上下列程式碼，填入先前定義的空白 `enableCam` 函式程式碼：

注意：請務必複製下列程式碼，並貼到上一個步驟定義的 `enableCam` 函式上，而不是只加到檔案結尾。

script.js

```
// Enable the live webcam view and start classification.
function enableCam(event) {
  // Only continue if the COCO-SSD has finished loading.
  if (!model) {
    return;
  }

  // Hide the button once clicked.
  event.target.classList.add('removed');

  // getUsermedia parameters to force video but not audio.
  const constraints = {
    video: true
  };

  // Activate the webcam stream.
  navigator.mediaDevices.getUserMedia(constraints).then(function(stream) {
    video.srcObject = stream;
    video.addEventListener('loadeddata', predictWebcam);
  });
}
```

最後，新增一些暫時性程式碼，測試網路攝影機是否正常運作。

以下程式碼會模擬載入模型並啟用攝影機按鈕，方便您點選。您將在下一個步驟中替換所有程式碼，因此請準備好稍後再次刪除：

script.js

```
// Placeholder function for next step.
function predictWebcam() {
}

// Pretend model has loaded so we can try out the webcam code.
var model = true;
demosSection.classList.remove('invisible');
```

試試看！此時，如果您實際執行目前的程式碼，並在按鈕可用時點選，您應該會看到開啟的網路攝影機即時畫面，前提是您已在網路瀏覽器中[接受網路攝影機的存取權](#) (通常會以彈出式視窗的形式顯示在瀏覽器左上角)。

太好了！如果您執行程式碼並點選目前的按鈕，畫面應如下所示：

Multiple object detection using pre trained model in TensorFlow.js

Wait for the model to load before clicking the button to enable the webcam - at which point it will become visible to use.

Hold some objects up close to your webcam to get a real-time classification! When ready click "enable webcam" below and accept access to the webcam when the browser asks (check the top left of your window)



8. 機器學習模型使用情況

載入模型

您現在可以載入 COCO-SSD 模型。

初始化完成後，請在網頁上啟用示範區域和按鈕 (將這段程式碼貼到您在上一部結尾加入的暫時程式碼上)：

script.js

```
// Store the resulting model in the global scope of our app.
var model = undefined;

// Before we can use COCO-SSD class we must wait for it to finish
// loading. Machine Learning models can be large and take a moment
// to get everything needed to run.
// Note: cocoSsd is an external object loaded from our index.html
// script tag import so ignore any warning in Glitch.
cocoSsd.load().then(function (loadedModel) {
  model = loadedModel;
  // Show demo section now model is ready to use.
  demosSection.classList.remove('invisible');
});
```

加入上述程式碼並重新整理即時監控畫面後，您會發現網頁載入幾秒後 (視網路速度而定)，模型準備就緒時會自動顯示「啟用網路攝影機」按鈕。不過，您也貼上了 `predictWebcam` 函式。現在是時候完整定義這項功能，因為目前我們的程式碼不會執行任何動作。

接著進行下一個步驟！

分類網路攝影機的畫面

執行下列程式碼，讓應用程式在瀏覽器準備就緒時，持續從網路攝影機串流擷取影格，並傳遞至模型進行分類。

接著，模型會剖析結果，並在傳回的座標位置繪製 `<p>` 標記，如果可信度達到特定程度，則將文字設為物件的標籤。

script.js

```

var children = [];

function predictWebcam() {
  // Now let's start classifying a frame in the stream.
  model.detect(video).then(function (predictions) {
    // Remove any highlighting we did previous frame.
    for (let i = 0; i < children.length; i++) {
      liveView.removeChild(children[i]);
    }
    children.splice(0);

    // Now lets loop through predictions and draw them to the live view if
    // they have a high confidence score.
    for (let n = 0; n < predictions.length; n++) {
      // If we are over 66% sure we are sure we classified it right, draw it!
      if (predictions[n].score > 0.66) {
        const p = document.createElement('p');
        p.innerText = predictions[n].class + ' - with '
          + Math.round(parseFloat(predictions[n].score) * 100)
          + '% confidence.';
        p.style = 'margin-left: ' + predictions[n].bbox[0] + 'px; margin-top: '
          + (predictions[n].bbox[1] - 10) + 'px; width: '
          + (predictions[n].bbox[2] - 10) + 'px; top: 0; left: 0;';

        const highlighter = document.createElement('div');
        highlighter.setAttribute('class', 'highlighter');
        highlighter.style = 'left: ' + predictions[n].bbox[0] + 'px; top: '
          + predictions[n].bbox[1] + 'px; width: '
          + predictions[n].bbox[2] + 'px; height: '
          + predictions[n].bbox[3] + 'px;';

        liveView.appendChild(highlighter);
        liveView.appendChild(p);
        children.push(highlighter);
        children.push(p);
      }
    }

    // Call this function again to keep predicting when the browser is ready.
    window.requestAnimationFrame(predictWebcam);
  });
}

```

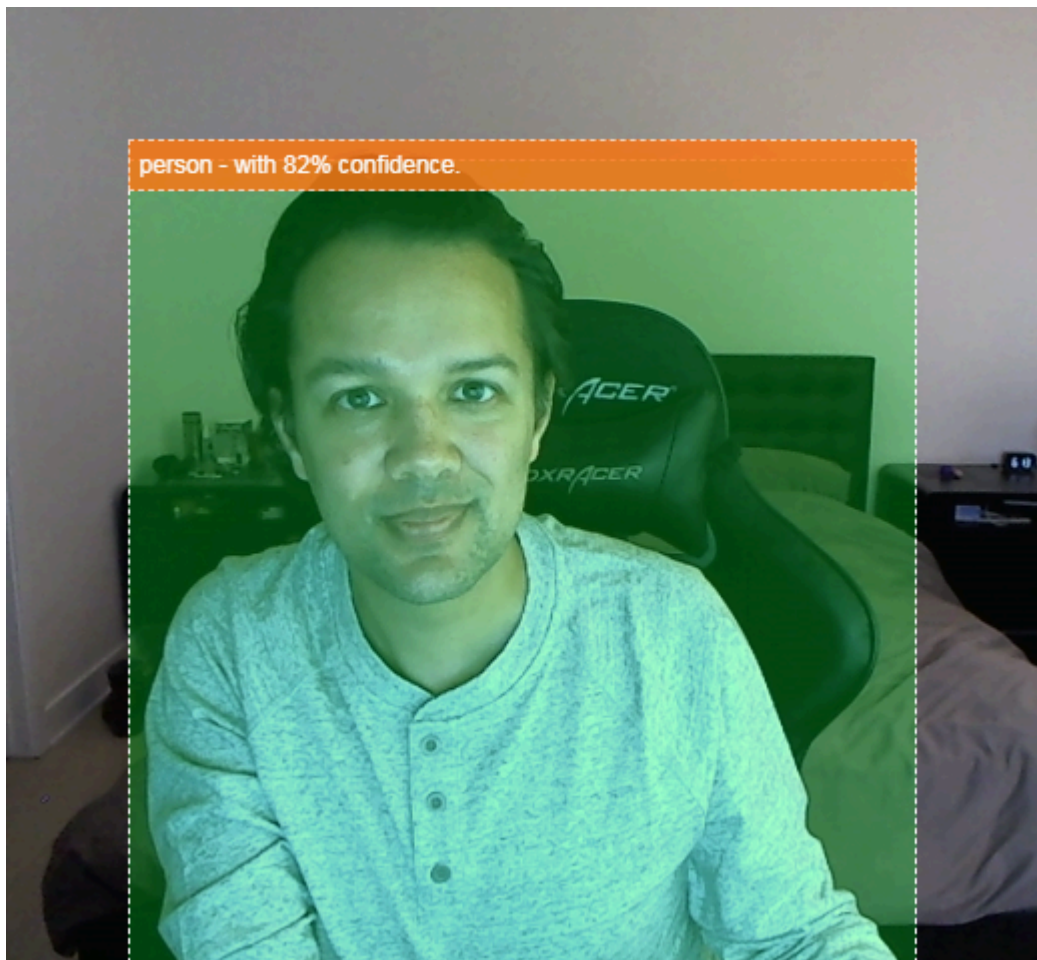
這段新程式碼中真正重要的呼叫是 *model.detect()*。

TensorFlow.js 的所有預先建構模型都有類似的函式 (名稱可能會因模型而異，詳情請參閱文件)，可實際執行機器學習推論。

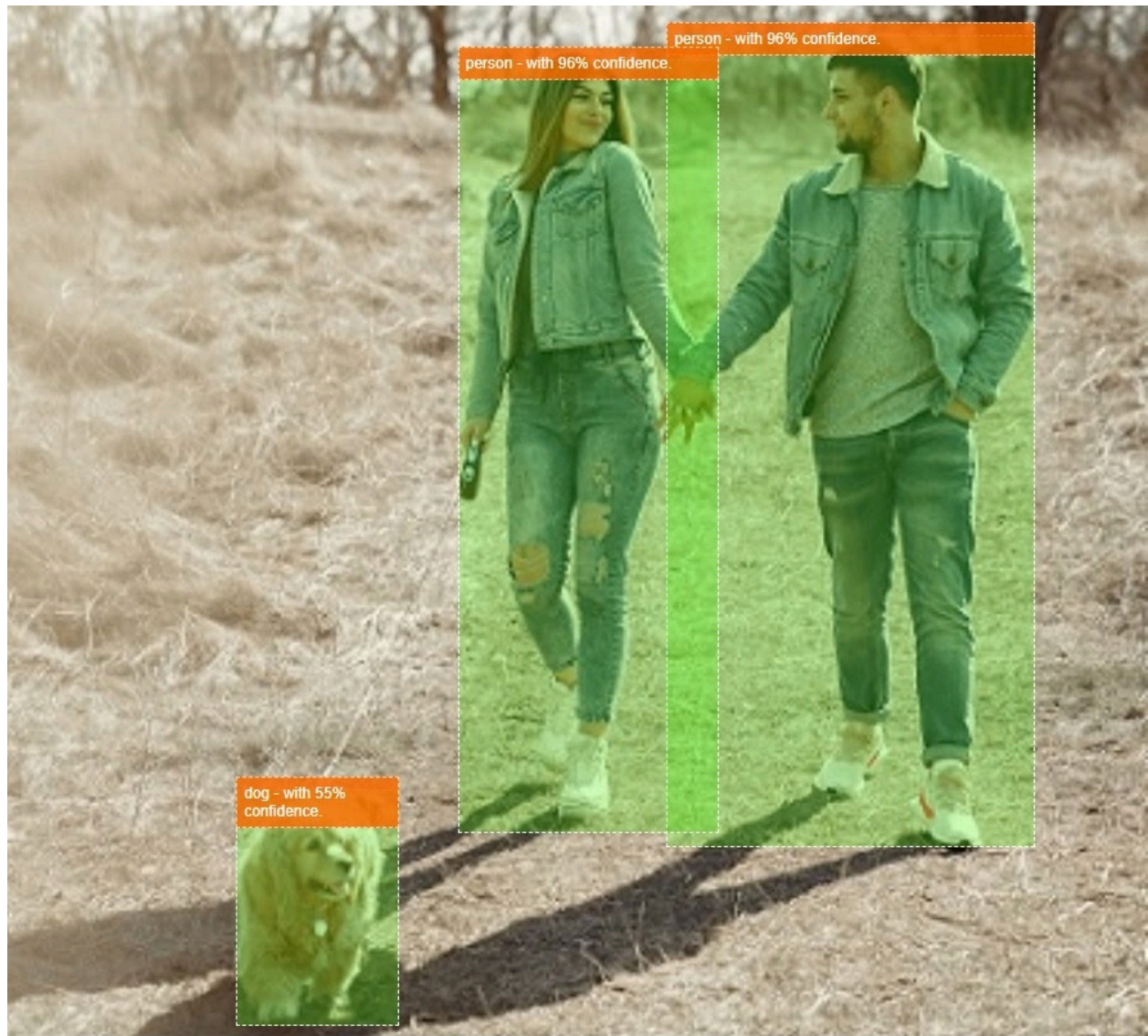
推論只是指取得一些輸入內容，並透過機器學習模型 (本質上是大量的數學運算) 執行，然後提供一些結果。我們使用 TensorFlow.js 預先建構的模型，以 JSON 物件的形式傳回預測結果，方便使用。

如要查看這項預測函式的完整詳細資料，請參閱 [GitHub 上的 COCO-SSD 模型說明文件](#)。這個函式會在幕後執行大量繁重工作：它可以接受任何「類似圖片」的物件做為參數，例如圖片、影片、畫布等。使用預先建構的模型可節省大量時間和精力，因為您不必自行編寫這段程式碼，即可「開箱即用」。

執行這段程式碼後，您應該會看到類似下圖的圖片：



最後，以下程式碼範例會同時偵測多個物件：



太厲害了！現在，您應該可以想像使用舊手機建立 Nest Cam 等裝置有多簡單，只要偵測到狗狗或貓咪在沙發上，就會發出通知。如果程式碼有問題，請[在這裡查看最終可運作的版本](#)，確認是否複製錯誤。

9. 恭喜

恭喜！您已踏出第一步，開始在網路瀏覽器中使用 TensorFlow.js 和機器學習技術。現在就換你接手，將這些簡單的基礎內容變成創意十足的影片。你想製作什麼？

重點回顧

在本程式碼研究室中，我們：

- 瞭解相較於其他形式的 TensorFlow，使用 TensorFlow.js 的優點。
- 瞭解在哪些情況下，您可能會想從預先訓練的機器學習模型著手。
- 建立可使用網路攝影機即時分類物件的完整網頁，包括：
- 為內容建立 HTML 架構
- 定義 HTML 元素和類別的樣式
- 設定 JavaScript 支架，與 HTML 互動並偵測網路攝影機是否存在
- 載入預先訓練的 TensorFlow.js 模型
- 使用載入的模型持續分類網路攝影機串流，並在圖片中的物件周圍繪製定界框。

後續步驟

歡迎與我們分享你的作品！您也可以輕鬆將這個程式碼研究室的內容擴展到其他創意用途。建議您跳脫框架思考，並在完成後繼續駭客松。

- [查看這個模型可辨識的所有物件](#)，並思考如何運用這些知識執行動作。現在，您可以擴充今天製作的內容，進一步實現哪些創意？

(或許您可以新增簡單的伺服器端層，在看到您選擇的特定物件時，[使用 WebSocket](#) 將通知傳送至其他裝置。這項功能可讓舊智慧型手機發揮新用途，(無限可能！)

- 在社群媒體上使用 **#MadeWithTFJS** 主題標記標記我們，就有機會讓你的專案登上 [TensorFlow 網誌](#)，甚至在日後的 TensorFlow 活動中展示。

更多 TensorFlow.js 程式碼研究室，深入瞭解相關主題

- [使用 TensorFlow.js 從頭開始編寫類神經網路](#)
- [在 TensorFlow.js 中使用遷移學習技術辨識音訊](#)
- [使用 TensorFlow.js 中的遷移學習功能進行自訂圖片分類](#)

值得一訪的網站

- [TensorFlow.js 官方網站](#)
 - [TensorFlow.js 預先建構模型](#)
 - [TensorFlow.js API](#)
 - [TensorFlow.js 示範](#)
-